

Doc 9.1 — Basic Loops: `for`, `while`, `do...while`

Section 9 — Loops & Iteration

1. Loop কী?

Programming-এ একই ধরনের কাজ বারবার করার প্রয়োজন হয়।

ধরো:

```
console.log("Hello");  
console.log("Hello");  
console.log("Hello");  
console.log("Hello");  
console.log("Hello");
```

এভাবে 100 বার লিখতে হলে অনেক unnecessary code হবে।

Loop ব্যবহার করে:

```
for (let i = 0; i < 5; i++) {  
  console.log("Hello");  
}
```

একই কাজ 5 বার করা যায়।

সহজ ভাষায়:

Loop = কোনো code নির্দিষ্ট condition অনুযায়ী বারবার execute করা।

2. কেন Loop ব্যবহার করি?

Loop সাধারণত ব্যবহার করি:

- Array-এর elements process করতে
- কোনো কাজ বারবার করতে
- সংখ্যা গণনা করতে

- data search করতে
- repeated calculation করতে
- API/data process করতে
- nested data process করতে
- নির্দিষ্ট condition পর্যন্ত কাজ করতে

Example:

```
const numbers = [10, 20, 30, 40];

for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]);
}
```

Output:

```
10
20
30
40
```

3. JavaScript-এর Basic Loop Types

এই doc-এ আমরা 3টা loop শিখব:

```
for
while
do...while
```

Basic difference

Loop	কখন ব্যবহার করব?
for	iteration count জানা থাকলে
while	condition-based repetition হলে
do...while	অন্তত একবার code execute করতেই হলে

4. **for** Loop

সবচেয়ে commonly used loop হলো:

```
for
```

Basic syntax:

```
for (initialization; condition; update) {  
  // code  
}
```

Example:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

Output:

```
0  
1  
2  
3  
4
```

5. **for** Loop-এর 3টি Part

এই অংশটা খুব ভালোভাবে বুঝতে হবে:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

এখানে:

```
let i = 0
```

→ Initialization

```
i < 5
```

→ Condition

```
i++
```

→ Update

6. Initialization

Initialization সাধারণত loop শুরু হওয়ার আগে একবার execute হয়।

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

এখানে:

```
let i = 0;
```

শুধু একবার execute হবে।

Mental model:

```
i = 0  
↓  
loop start
```

7. Condition

Condition decide করে loop চলবে কি না।

```
i < 5
```

যতক্ষণ:

```
i < 5
```

true থাকবে, loop চলবে।

যখন:

```
i >= 5
```

হয়ে যাবে, loop stop করবে।

8. Update

প্রতিটি iteration-এর শেষে update execute হয়।

```
i++
```

মানে:

```
i = i + 1;
```

So:

```
0  
1  
2  
3  
4  
5
```

5 এ গিয়ে condition:

```
5 < 5
```

false।

তাই loop stop।

9. `for` Loop Execution Flow

এই loop:

```
for (let i = 0; i < 3; i++) {  
    console.log(i);  
}
```

এভাবে execute হয়:

```
1. let i = 0  
    ↓  
2. i < 3 ? → true  
    ↓  
3. console.log(i)  
    ↓  
4. i++  
    ↓  
5. i < 3 ? → true  
    ↓  
6. console.log(i)  
    ↓  
7. i++  
    ↓  
8. i < 3 ? → true  
    ↓  
9. console.log(i)  
    ↓  
10. i++  
    ↓  
11. i < 3 ? → false  
    ↓  
12. STOP
```

এটাই `for` loop-এর core mechanism।

10. `i++` কী?

```
i++;
```

মান্নে:

```
i = i + 1;
```

Example:

```
let i = 0;

i++;
console.log(i);
```

Output:

```
1
```

11. `i--`

Reverse counting-এর জন্ম:

```
i--;
```

Example:

```
for (let i = 5; i > 0; i--) {
  console.log(i);
}
```

Output:

```
5
4
3
2
1
```

12. `++i` vs `i++`

দুটোই value 1 বাড়ায়।

```
i++;
```

এবং:

```
++i;
```

কিন্তু expression-এর ভেতরে এদের return behavior আলাদা।

Example:

```
let i = 5;

console.log(i++);
console.log(i);
```

Output:

```
5
6
```

`i++` আগে old value return করে, তারপর increment করে।

Example:

```
let i = 5;

console.log(++i);
console.log(i);
```

Output:

```
6
6
```

`++i` আগে increment করে, তারপর new value return করে।

Loop-এর ক্ষেত্রে

সাধারণত:

```
i++
```

বা:

```
i--
```

দেখলেই যথেষ্ট।

13. Simple Counting

1 থেকে 10:

```
for (let i = 1; i <= 10; i++) {  
  console.log(i);  
}
```

Output:

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

14. Even Numbers

```
for (let i = 2; i <= 10; i += 2) {  
  console.log(i);  
}
```

Output:

```
2  
4  
6  
8  
10
```

এখানে:

```
i += 2;
```

মানে:

```
i = i + 2;
```

15. Odd Numbers

```
for (let i = 1; i <= 10; i += 2) {  
  console.log(i);  
}
```

Output:

```
1  
3  
5  
7  
9
```

16. Reverse Loop

```
for (let i = 10; i >= 1; i--) {  
  console.log(i);  
}
```

Output:

```
10  
9  
8  
7  
6  
5  
4  
3  
2  
1
```

17. Custom Step

Loop সবসময় 1 করে increment করতে হবে এমন না।

```
for (let i = 0; i <= 20; i += 5) {  
  console.log(i);  
}
```

Output:

```
0  
5  
10  
15  
20
```

18. Array Traversal with `for`

Traditional `for` loop দিয়ে array traverse করা যায়।

```
const fruits = ["Apple", "Banana", "Mango"];

for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

Output:

```
Apple
Banana
Mango
```

কেন `i < fruits.length`?

Array-এর length:

```
3
```

Valid indexes:

```
0
1
2
```

তাই:

```
i < fruits.length
```

মানে:

```
i < 3
```

So `i` হবে:

```
0 → 1 → 2
```

19. Common Array Loop Pattern

এই patternটা খুব important:

```
for (let i = 0; i < array.length; i++) {  
    console.log(array[i]);  
}
```

এটা memorize করার চেয়ে structure বুঝে রাখো:

```
start → 0  
condition → i < array.length  
update → i++  
access → array[i]
```

20. Modifying Array Elements

Loop দিয়ে array-এর elements update করা যায়।

```
const numbers = [1, 2, 3, 4];  
  
for (let i = 0; i < numbers.length; i++) {  
    numbers[i] = numbers[i] * 2;  
}  
  
console.log(numbers);
```

Output:

```
[2, 4, 6, 8]
```

21. Sum of Array

```
const numbers = [10, 20, 30, 40];

let sum = 0;

for (let i = 0; i < numbers.length; i++) {
  sum += numbers[i];
}

console.log(sum);
```

Output:

```
100
```

Mental model

```
sum = 0

+ 10 → 10
+ 20 → 30
+ 30 → 60
+ 40 → 100
```

এখানে `sum` হচ্ছে accumulator।

Later `reduce()` শেখার সময় এই concept আবার আসবে।

22. Finding an Element

```
const numbers = [10, 20, 30, 40];

for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] === 30) {
    console.log("Found");
  }
}
```

Output:

```
Found
```

আরও efficientভাবে early exit করতে পরে `break` ব্যবহার করতে পারবে।

23. `for` Loop Variable Scope

```
for (let i = 0; i < 3; i++) {  
  console.log(i);  
}
```

Loop শেষ হওয়ার পর:

```
console.log(i);
```

করলে error হবে।

কারণ `let i` loop block-এর মধ্যে scoped।

24. `var` vs `let` in Loop

`var` function-scoped:

```
for (var i = 0; i < 3; i++) {  
  console.log(i);  
}  
  
console.log(i);
```

Output:

```
3
```

কিন্তু:

```
for (let i = 0; i < 3; i++) {  
  console.log(i);  
}  
  
console.log(i);
```

এখানে `i` loop block-এর বাইরে accessible না।

Modern JavaScript-এ সাধারণত loop counter-এর জন্য:

```
let
```

use করাই better।

25. `while` Loop

`while` loop syntax:

```
while (condition) {  
  // code  
}
```

Example:

```
let i = 0;  
  
while (i < 5) {  
  console.log(i);  
  
  i++;  
}
```

Output:

```
0  
1  
2  
3  
4
```

26. **while** Loop কীভাবে কাজ করে?

while loop প্রথমে condition check করে।

```
condition?  
  ↓  
true → execute  
  ↓  
update  
  ↓  
condition?
```

Condition false হলে loop stop।

27. **for** vs **while**

Same কাজ:

for

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

while

```
let i = 0;  
  
while (i < 5) {  
  console.log(i);  
  
  i++;  
}
```

দুটোর output same।

Difference mainly code structure এবং use case-এ।

28. কখন `while` ব্যবহার করব?

যখন exact কতবার loop চলবে সেটা আগে থেকে জানা নেই, তখন `while` useful।

Example:

```
while (userHasNotFinished) {  
    // keep working  
}
```

আরেকটা example:

```
while (password !== correctPassword) {  
    // ask again  
}
```

তবে বাস্তব application-এ user input handling সাধারণত UI/event-based হয়; infinite synchronous `while` loop browser UI freeze করতে পারে।

29. `while` Loop — User Input Concept

Conceptually:

```
let attempts = 0;  
  
while (attempts < 3) {  
    console.log("Try again");  
  
    attempts++;  
}
```

Output:

```
Try again  
Try again  
Try again
```

30. **while** Loop-এর সবচেয়ে Common Mistake

Update ভুলে যাওয়া।

✗:

```
let i = 0;

while (i < 5) {
  console.log(i);
}
```

এখানে:

```
i = 0
```

থেকেই যাবে।

Condition:

```
0 < 5
```

সবসময় true।

Result:

Infinite loop

Correct:

```
let i = 0;

while (i < 5) {
  console.log(i);

  i++;
}
```

31. Infinite Loop

যে loop কখনো terminate করে না তাকে infinite loop বলা হয়।

Example:

```
while (true) {  
    console.log("Hello");  
}
```

এটা theoretically forever চলবে যতক্ষণ না execution interrupt করা হয়।

আরেকটা:

```
for (let i = 0; i < 10; ) {  
    console.log(i);  
}
```

এখানে `i` update হচ্ছে না।

তাই infinite loop।

Important

Infinite loop সবসময় bad না।

কিছু systems/programming patterns-এ intentional infinite loops থাকে, কিন্তু তাদের controlled exit mechanism থাকতে হয়।

32. `do...while` Loop

Syntax:

```
do {  
    // code  
} while (condition);
```

এখানে special feature হলো:

Condition check করার আগে code অন্তত একবার execute হবে।

Example:

```
let i = 0;

do {
  console.log(i);

  i++;
} while (i < 5);
```

Output:

```
0
1
2
3
4
```

33. **while** vs **do...while**

এটাই main difference।

while

```
let i = 10;

while (i < 5) {
  console.log(i);
}
```

Output:

```
Nothing
```

কারণ condition শুরুতেই false।

do...while

```
let i = 10;

do {
  console.log(i);
} while (i < 5);
```

Output:

```
10
```

কারণ `do` block একবার আগে execute হয়।

34. Execution Flow

while

```
condition
  ↓
false → STOP
  ↓
true
  ↓
execute
  ↓
update
  ↓
condition
```

do...while

```
execute
  ↓
update
  ↓
condition
  ↓
true → repeat
false → STOP
```

35. Real-World do...while Concept

ধরো menu অন্তত একবার দেখাতে হবে:

```
let choice;

do {
  console.log("1. Start");
  console.log("2. Exit");

  // get choice

} while (choice !== 2);
```

এখানে menu অন্তত একবার show হবে।

Modern frontend apps-এ অবশ্য UI interaction সাধারণত event handlers/state দিয়ে handle করা হয়; do...while বেশি useful algorithmic/console-style logic বোঝার জন্য।

36. `for` vs `while` vs `do...while`

Feature	<code>for</code>	<code>while</code>	<code>do...while</code>
Condition আগে check?	✓	✓	✗
অন্তত একবার চলে?	✗	✗	✓
Counter-based	Excellent	Good	Less common
Condition-based	Good	Excellent	Good
Initialization/update একই জায়গায়	✓	✗	✗
Common usage	Very common	Common	Less common

37. Same Problem — Three Ways

Print 1–5.

`for`

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

`while`

```
let i = 1;  
  
while (i <= 5) {  
  console.log(i);  
  
  i++;  
}
```

do...while

```
let i = 1;

do {
  console.log(i);

  i++;
} while (i <= 5);
```

তিনটার output:

```
1
2
3
4
5
```

38. Nested Loops

একটা loop-এর ভিতরে আরেকটা loop থাকলে সেটাকে nested loop বলে।

Example:

```
for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
    console.log(i, j);
  }
}
```

Output:

```
1 1
  1 2
  1 3
2 1
  2 2
  2 3
3 1
  3 2
  3 3
```

39. Nested Loop কীভাবে কাজ করে?

Outer loop:

```
i = 1
```

তখন inner loop পুরোটা চলে:

```
j = 1
  j = 2
  j = 3
```

তারপর:

```
i = 2
```

আবার inner loop পুরোটা চলে।

So:

```
i=1 → j=1, 2, 3
i=2 → j=1, 2, 3
i=3 → j=1, 2, 3
```

40. Nested Loop — Multiplication Table

```
for (let i = 1; i <= 5; i++) {  
  for (let j = 1; j <= 5; j++) {  
    console.log(i * j);  
  }  
}
```

এটা multiplication combinations তৈরি করবে।

41. Nested Loop — 2D Array

```
const matrix = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9]  
];  
  
for (let row = 0; row < matrix.length; row++) {  
  for (let col = 0; col < matrix[row].length; col++) {  
    console.log(matrix[row][col]);  
  }  
}
```

Output:

```
1  
2  
3  
4  
5  
6  
7  
8  
9
```

42. Nested Loop Complexity

যদি:

```
outer loop → n times  
inner loop → n times
```

তাহলে মোট operations approximately:

```
n × n = n2
```

So time complexity:

```
O(n2)
```

Example:

```
for (let i = 0; i < n; i++) {  
  for (let j = 0; j < n; j++) {  
    // work  
  }  
}
```

→ O(n²)

DSA শেখার সময় এই concept খুব important হবে।

43. Three Nested Loops

```
for (let i = 0; i < n; i++) {  
  for (let j = 0; j < n; j++) {  
    for (let k = 0; k < n; k++) {  
      // work  
    }  
  }  
}
```

Approximately:

$n \times n \times n$

So:

$O(n^3)$

Nested loop দেখলেই automatically $O(n^2)$ হবে এমন না; nesting কত level এবং loops-এর bounds কী, সেটা দেখতে হবে।

44. Loop দিয়ে Maximum বের করা

```
const numbers = [10, 50, 20, 80, 30];

let max = numbers[0];

for (let i = 1; i < numbers.length; i++) {
  if (numbers[i] > max) {
    max = numbers[i];
  }
}

console.log(max);
```

Output:

80

Mental model:

```
max = 10

50 > 10 → max = 50
20 > 50 → no
80 > 50 → max = 80
30 > 80 → no
```

45. Loop দিয়ে Minimum বের করা

```
const numbers = [10, 50, 20, 80, 30];

let min = numbers[0];

for (let i = 1; i < numbers.length; i++) {
  if (numbers[i] < min) {
    min = numbers[i];
  }
}

console.log(min);
```

Output:

```
10
```

46. Count Condition Matching Elements

```
const numbers = [10, 15, 20, 25, 30];

let count = 0;

for (let i = 0; i < numbers.length; i++) {
  if (numbers[i] % 2 === 0) {
    count++;
  }
}

console.log(count);
```

Output:

```
3
```

কারণ even numbers:

```
10
  20
  30
```

47. Reverse an Array Conceptually

```
const numbers = [1, 2, 3, 4, 5];

for (let i = numbers.length - 1; i >= 0; i--) {
  console.log(numbers[i]);
}
```

Output:

```
5
4
3
2
1
```

এখানে array modify করছি না; শুধু reverse direction-এ traverse করছি।

48. Loop with String

String-এর character index দিয়েও access করা যায়।

```
const name = "Shariar";

for (let i = 0; i < name.length; i++) {
  console.log(name[i]);
}
```

Output:

```
S  
h  
a  
r  
i  
a  
r
```

Later `for...of` দিয়ে এটা আরও cleanভাবে করা যাবে।

49. Loop দিয়ে String Count

কতগুলো vowel আছে:

```
const text = "javascript";  
  
let count = 0;  
  
for (let i = 0; i < text.length; i++) {  
  if ("aeiou".includes(text[i])) {  
    count++;  
  }  
}  
  
console.log(count);
```

Output:

```
3
```

50. Block Scope and Loop

Modern JavaScript-এ:

```
for (let i = 0; i < 3; i++) {  
  const message = `Number: ${i}`;  
  
  console.log(message);  
}
```

`i` এবং `message` loop block-এর ভিতরে scoped।
তাই বাইরে:

```
console.log(message);
```

করলে error হবে।

51. Empty Loop

Technically loop-এর body empty হতে পারে:

```
for (let i = 0; i < 5; i++);
```

এখানে semicolon-এর কারণে loop body empty।

এটা confusing হতে পারে।

❌ Avoid:

```
for (let i = 0; i < 5; i++);  
{  
  console.log(i);  
}
```

এই code structure সাধারণত bug তৈরি করবে।

52. Semicolon Mistake

❌:

```
for (let i = 0; i < 5; i++); {  
  console.log(i);  
}
```

এই semicolon:

```
;
```

loop-এর body শেষ করে দিয়েছে।

Best practice:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

53. Off-by-One Error

Loop-এর common bug:

```
for (let i = 0; i <= array.length; i++) {  
  console.log(array[i]);  
}
```

Problem:

যদি length হয়:

```
5
```

valid indexes:

```
0 1 2 3 4
```

কিন্তু `i <= 5` হলে `i = 5`-এ যাবে।

```
array[5]
```

→ `undefined`

Better:

```
for (let i = 0; i < array.length; i++) {  
  console.log(array[i]);  
}
```

Golden rule

Array index traversal-এর ক্ষেত্রে সাধারণত `i < array.length`।

54. Accidental Infinite Loop

✗:

```
let i = 0;  
  
while (i < 10) {  
  console.log(i);  
  
  i--;  
}
```

এখানে `i` বাড়ার বদলে কমছে।

So:

```
0  
-1  
-2  
-3  
...
```

এবং:

```
i < 10
```

সবসময় true থাকবে।

55. Wrong Update Direction

Condition:

```
i < 10
```

সাধারণত counter বাড়বে:

```
i++;
```

Condition:

```
i > 0
```

সাধারণত counter কমবে:

```
i--;
```

Mental check:

```
condition-এর দিকে progress হচ্ছে তো?
```

56. Avoid Unnecessary Nested Loops

যদি:

```
for (...) {  
    for (...) {  
        // expensive work  
    }  
}
```

নিখো, নিজেকে জিজ্ঞেস করো:

এটা কি সত্যিই দরকার?

অনেক DSA problem-এ nested loop unavoidable, কিন্তু কিছু ক্ষেত্রে data structure বা better algorithm দিয়ে

$O(n^2)$ থেকে $O(n)$ করা যায়।

57. Loop Selection Guide

Use **for** when:

কতবার iterate করবে জানা আছে

Example:

```
for (let i = 0; i < 10; i++) {  
  console.log(i);  
}
```

Use **while** when:

condition-based repetition

Example:

```
while (balance > 0) {  
  // process  
}
```

Use **do...while** when:

code অন্তত একবার execute করতেই হবে

Example:

```
do {  
  // show menu  
} while (choice !== "exit");
```

58. Modern JavaScript Note

Traditional `for` loop খুব important হলেও array iteration-এর জন্য JavaScript-এ আরও অনেক tools আছে:

```
for
  for...of
  forEach
  map
  filter
  reduce
  find
  some
  every
```

আমরা এগুলো আলাদা docs-এ already/পরবর্তীতে cover করছি।

বিশেষ করে:

```
for...of
```

এই section-এর **Doc 9.2**-তে আসবে।

59. `for` vs `forEach` Preview

Traditional:

```
for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]);
}
```

Later:

```
numbers.forEach((number) => {
  console.log(number);
});
```

দুটোর use case এক নয়।

`for` loop-এর একটা বড় সুবিধা হলো loop flow-এর ওপর বেশি direct control পাওয়া—যেমন `break` / `continue` ব্যবহার করা।

60. Important Loop Patterns

JavaScript/DSA-তে এই patterns বারবার আসবে:

Pattern 1 — Traverse

```
for (let i = 0; i < arr.length; i++) {  
    // process arr[i]  
}
```

Pattern 2 — Reverse Traverse

```
for (let i = arr.length - 1; i >= 0; i--) {  
    // process arr[i]  
}
```

Pattern 3 — Accumulator

```
let sum = 0;  
  
for (let i = 0; i < arr.length; i++) {  
    sum += arr[i];  
}
```

Pattern 4 — Counter

```
let count = 0;  
  
for (let i = 0; i < arr.length; i++) {  
    if (condition) {  
        count++;  
    }  
}
```

Pattern 5 — Find Maximum

```
let max = arr[0];

for (let i = 1; i < arr.length; i++) {
  if (arr[i] > max) {
    max = arr[i];
  }
}
```

61. Practice — Output Prediction

Question 1

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

Output কী হবে?

Question 2

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

Question 3

```
for (let i = 5; i > 0; i--) {
  console.log(i);
}
```

Question 4

```
for (let i = 0; i <= 10; i += 2) {  
  console.log(i);  
}
```

Question 5

```
let i = 0;  
  
while (i < 3) {  
  console.log(i);  
  i++;  
}
```

Question 6

```
let i = 10;  
  
do {  
  console.log(i);  
} while (i < 5);
```

Question 7

```
const arr = ["A", "B", "C"];  
  
for (let i = 0; i < arr.length; i++) {  
  console.log(arr[i]);  
}
```

62. Coding Challenge 1 — Sum

এই array:

```
const numbers = [5, 10, 15, 20, 25];
```

একটা `for` loop ব্যবহার করে সব সংখ্যার sum বের করো।

Expected:

```
75
```

63. Coding Challenge 2 — Even Numbers

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

`for` loop ব্যবহার করে শুধু even numbers print করো।

Expected:

```
2
4
6
8
10
```

64. Coding Challenge 3 — Maximum

```
const numbers = [25, 10, 80, 45, 60];
```

Loop ব্যবহার করে maximum value বের করো।

Expected:

```
80
```

65. Coding Challenge 4 — Count Vowels

```
const text = "javascript";
```

Loop ব্যবহার করে vowel count বের করো।

Expected:

```
3
```

66. Coding Challenge 5 — Reverse Print

```
const numbers = [10, 20, 30, 40, 50];
```

Array reverse না করে loop ব্যবহার করে reverse order-এ print করো।

Expected:

```
50
40
30
20
10
```

67. Coding Challenge 6 — Nested Loop

Nested loop ব্যবহার করে এই pattern print করার চেষ্টা করো:

```
*
**
***
****
*****
```

Hint:

Outer loop → rows

68. Interview Questions

Beginner

1. What is a loop?
 2. Why do we use loops?
 3. What are the three basic JavaScript loops?
 4. What are the three parts of a `for` loop?
 5. What does `i++` mean?
 6. What is the difference between `i++` and `i--`?
 7. What is an infinite loop?
 8. What is the difference between `while` and `do...while`?
-

Intermediate

9. Why does array traversal usually use `i < array.length`?
 10. What is an off-by-one error?
 11. When would you prefer `while` over `for`?
 12. Why does `do...while` always execute at least once?
 13. What happens if you forget to update the loop variable?
 14. What is a nested loop?
 15. What is the time complexity of two independent nested loops over `n` elements?
-

Advanced

16. Explain the execution order of a `for` loop.
 17. Why is `let` generally preferred over `var` for loop counters?
 18. How can a nested loop result in $O(n^2)$?
 19. When can a nested loop still be less than $O(n^2)$?
 20. How would you optimize an unnecessarily nested loop?
 21. What is the difference between loop traversal and array transformation?
 22. When would traditional `for` be preferable to `forEach`?
-

69. Quick Revision

Concept	Key Point
<code>for</code>	Count/control based loop
<code>while</code>	Condition-based loop
<code>do...while</code>	Executes at least once
Initialization	Loop শুরু করার setup
Condition	Loop চলবে কি না
Update	Counter/state পরিবর্তন
<code>i++</code>	<code>i = i + 1</code>
<code>i--</code>	<code>i = i - 1</code>
Nested loop	Loop-এর ভিতরে loop
Infinite loop	Termination condition কখনো false হয় না
<code>array.length</code>	Array-এর total elements
<code>array[i]</code>	Current element
<code>i < array.length</code>	Safe common traversal condition
<code>O(n²)</code>	সাধারণ two-level nested iteration

70. Final Mental Model

Loop শেখার সময় এই 4টা প্রশ্ন করো:

1. কোথা থেকে শুরু করব?

↓

Initialization

2. কখন পর্যন্ত চলবে?

↓

Condition

3. প্রতি iteration-এর পরে কী পরিবর্তন হবে?

↓

Update

4. প্রতি iteration-এ কী কাজ হবে?

↓

Loop Body

for :

Initialize

↓

Condition

↓

Body

↓

Update

↓

Condition

↓

...

while :

Condition

↓

Body

↓

Update

↓

Condition

↓

...

`do...while`:

Body

↓

Update

↓

Condition

↓

Repeat / Stop

Golden Rules

1. `for` → iteration count জানা থাকলে খুব useful।
2. `while` → condition-based repetition-এর জন্য useful।
3. `do...while` → body অন্তত একবার execute হবে।
4. Array traversal-এ সাধারণ pattern:

```
for (let i = 0; i < arr.length; i++) {  
    // arr[i]  
}
```

5. `i < arr.length` সাধারণত `i <= arr.length` -এর চেয়ে correct।
6. Loop variable update করতে ভুললে infinite loop হতে পারে।
7. Nested loops algorithm-এর time complexity বাড়াতে পারে।
8. `let` loop counter-এর জন্য modern JavaScript-এ সাধারণত better choice।
9. Reverse traversal:

```
for (let i = arr.length - 1; i >= 0; i--) {  
    // ...  
}
```

10. Loop-এর code শুধু repeat না—**traversal, searching, counting, accumulation, comparison**-এর foundation।

Doc 9.1 Mastery Checklist

- ☐ Loop কী এবং কেন ব্যবহার করি
- ☐ `for` loop syntax

- ☐ Initialization
- ☐ Condition
- ☐ Update
- ☐ `i++`
- ☐ `i--`
- ☐ `++i` vs `i++`
- ☐ Forward loop
- ☐ Reverse loop
- ☐ Custom increment
- ☐ Array traversal
- ☐ String traversal
- ☐ Array modification
- ☐ Sum calculation
- ☐ Counting
- ☐ Min/Max finding
- ☐ `while`
- ☐ `do...while`
- ☐ Infinite loops
- ☐ Off-by-one errors
- ☐ Nested loops
- ☐ Basic loop complexity
- ☐ Loop selection



Next — Doc 9.2

পরের doc-এ আসবে:

Doc 9.2 – for...of & for...in

for...of

- └─ Arrays
- └─ Strings
- └─ Set
- └─ Map
- └─ Iterables
- └─ Values

for...in

- └─ Object keys
- └─ Object iteration
- └─ Own vs inherited properties
- └─ Object.keys()
- └─ কেন Array-এর জন্য for...in avoid করা হয়

সবচেয়ে important line:

for...of → VALUES

for...in → KEYS